

Manual Técnico - GoLite IDE

Proyecto: GoLite - Fase 2

Curso: Organización de Lenguajes y Compiladores 1

Estudiante: Dalio Antonio Miranda Guerra

Carnet: 202100116

Universidad: Universidad de San Carlos de Guatemala - Facultad de Ingeniería

1. Descripción técnica

GoLite IDE es un intérprete desarrollado en Java para el lenguaje GoLite. El sistema incluye una interfaz gráfica, un analizador léxico generado con JFlex, un analizador sintáctico generado con CUP, construcción de AST, análisis semántico, interpretación y generación de reportes.

El objetivo principal es procesar código fuente `.glt`, validar su estructura, construir un Árbol de Sintaxis Abstracta y ejecutar las instrucciones definidas por el usuario.

2. Tecnologías utilizadas

- Java JDK 21.
 - Maven.
 - JFlex.
 - CUP.
 - Java Swing.
 - Graphviz.
-

3. Estructura general del proyecto

```
GoLite/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── analisis/
│   │   │   └── com/golite/
│   │   │       ├── ast/
│   │   │       ├── gui/
│   │   │       ├── interpreter/
│   │   │       └── reports/
│   │   └── jflex/
│   └── reportes/
```

```
|   |— ast.dot
|   |— ast.png
|— target/
|— pom.xml
|— README.md
|— Manual_Usuario.md
|— Manual_Tecnico.md
|— Gramatica.bnf
```

4. Flujo general del sistema

El flujo del intérprete es el siguiente:

1. El usuario escribe o abre un archivo `.glt`.
2. El contenido se envía al analizador léxico.
3. El analizador léxico genera tokens.
4. El analizador sintáctico valida la estructura gramatical.
5. Se construye el AST.
6. El intérprete recorre el AST.
7. Se ejecutan instrucciones y expresiones.
8. Se registran errores si existen.
9. Se muestran resultados en consola.
10. Se generan reportes desde la interfaz.

5. Analizador léxico

El analizador léxico está definido con JFlex.

Su función es reconocer:

- Palabras reservadas.
- Identificadores.
- Números enteros.
- Números decimales.
- Cadenas.
- Runes.
- Operadores.
- Signos de agrupación.
- Comentarios.
- Caracteres no reconocidos.

El scanner genera una lista de tokens para el reporte de tokens.

También almacena errores léxicos cuando encuentra caracteres inválidos.

Ejemplo de error léxico:

```
func main() {  
  @  
}
```

6. Analizador sintáctico

El analizador sintáctico está implementado con CUP.

Su función es validar que los tokens generados por el scanner cumplan con la gramática del lenguaje GoLite.

El parser genera como resultado un AST representado por nodos Java.

El símbolo inicial del proyecto es el programa completo, que contiene declaraciones globales, funciones y structs.

7. AST

El AST está formado por clases ubicadas en el paquete:

```
com.golite.ast
```

Cada nodo representa una construcción del lenguaje.

Ejemplos de nodos:

- ProgramNode
- FunctionNode
- VarDeclNode
- AssignNode
- BinaryOpNode
- IfNode
- ForNode
- SwitchNode
- ReturnNode
- SliceLiteralNode
- SliceAccessNode

- StructDeclNode
- StructInitNode
- StructAccessNode
- StructAssignNode

Cada nodo implementa el método:

```
accept(Visitor visitor, Environment env)
```

Este método permite aplicar el patrón Visitor para recorrer el AST.

8. Patrón Visitor

El proyecto utiliza el patrón Visitor para separar la estructura del AST de las operaciones que se realizan sobre él.

La interfaz `Visitor` define métodos para visitar cada tipo de nodo.

Ejemplo conceptual:

```
Object visit(VarDeclNode node, Environment env);  
Object visit(BinaryOpNode node, Environment env);  
Object visit(StructAssignNode node, Environment env);
```

El intérprete implementa esta interfaz y define la lógica de ejecución de cada nodo.

9. Intérprete

El intérprete se encuentra en el paquete:

```
com.golite.interpreter
```

Su clase principal es:

```
Interpreter
```

El intérprete se encarga de:

- Ejecutar instrucciones.

- Evaluar expresiones.
- Manejar variables.
- Ejecutar funciones.
- Validar tipos.
- Registrar errores semánticos.
- Manejar slices.
- Manejar matrices.
- Manejar structs.
- Ejecutar funciones embebidas.

10. Entornos y ámbitos

El manejo de ámbitos se realiza mediante la clase:

Environment

Cada entorno almacena símbolos declarados en un ámbito específico.

El sistema permite:

- Variables globales.
- Variables locales.
- Bloques independientes.
- Ámbitos dentro de funciones.
- Búsqueda en entornos padre.

Esto permite validar errores como acceder a una variable fuera de su ámbito.

Ejemplo:

```
func main() {  
    {  
        y := 20  
    }  
  
    fmt.Println(y)  
}
```

El intérprete genera un error semántico porque `y` no existe fuera del bloque.

11. Tabla de símbolos

La tabla de símbolos se genera recorriendo el AST.

El reporte puede incluir:

- Variables.
- Funciones.
- Structs.
- Tipo.
- Categoría.
- Ámbito.
- Línea.
- Columna.

El reporte se muestra desde la interfaz gráfica mediante una tabla.

12. Manejo de errores

El sistema maneja tres tipos principales de errores:

Errores léxicos

Se generan cuando el scanner encuentra caracteres no reconocidos.

Errores sintácticos

Se generan cuando el parser encuentra una estructura inválida.

Errores semánticos

Se generan durante la interpretación.

Ejemplos:

- Variable no declarada.
- Tipo incompatible.
- Operación inválida.
- Índice fuera de rango.
- Acceso a un valor que no es slice.
- Atributo inexistente en struct.

Los errores se muestran en la consola y en el reporte de errores.

13. Slices

Los slices se representan internamente usando listas dinámicas de Java.

Funcionalidades implementadas:

- Creación de slice.
- Acceso por índice.
- `len`.
- `append`.
- `slices.Index`.

Ejemplo:

```
nums := []int{10,20,30}
nums = append(nums,40)
fmt.Println(nums[3])
```

14. Matrices

Las matrices se implementan como slices de slices.

Ejemplo:

```
fila1 := []int{1,2}
fila2 := []int{3,4}

matriz := [][]int{fila1,fila2}

fmt.Println(matriz[0][0])
fmt.Println(matriz[1][1])
```

Internamente, una matriz es una lista donde cada elemento también es una lista.

15. Structs

Los structs se implementan mediante mapas de Java.

Cada instancia de struct almacena sus atributos como pares clave-valor.

Ejemplo:

```
type Persona struct {  
    nombre string, edad int  
}  
  
p := Persona{nombre: "Dalio", edad: 24}
```

Internamente, la instancia puede representarse como:

```
HashMap<String, Object>
```

Además, se guarda el nombre del tipo del struct para validar funciones que reciben structs como parámetros.

16. Acceso y asignación de atributos

El acceso a atributos se realiza con:

```
p.nombre
```

La modificación de atributos se realiza con:

```
p.nombre = "Carlos"
```

El nodo encargado de esta modificación es:

```
StructAssignNode
```

Este nodo contiene:

- Nombre de la variable struct.
- Nombre del atributo.
- Valor nuevo.

El intérprete valida:

1. Que el struct exista.
 2. Que la variable sea una instancia de struct.
 3. Que el atributo exista.
 4. Que pueda actualizarse el valor.
-

17. Funciones con structs

El sistema permite funciones que reciben structs como parámetros.

Ejemplo:

```
func cambiarNombre(p Persona, nuevo string) {  
    p.nombre = nuevo  
}
```

Para permitir esto, la gramática acepta `ID` como tipo.

Ejemplo:

```
<tipo> ::= "int"  
        | "float64"  
        | "string"  
        | "bool"  
        | "rune"  
        | ID
```

Cuando una función recibe un struct, el intérprete valida que el tipo del argumento coincida con el tipo esperado.

Debido a que los structs se manejan internamente como referencias, los cambios realizados dentro de la función se mantienen después de finalizar la llamada.

18. Funciones embebidas

El intérprete implementa las siguientes funciones embebidas:

- `fmt.Println`
- `strconv.Atoi`
- `strconv.ParseFloat`
- `reflect.TypeOf`
- `len`
- `append`
- `slices.Index`
- `strings.Join`

Estas funciones se manejan directamente dentro del intérprete.

19. Reporte AST

El reporte AST se genera mediante la clase:

```
ASTReport
```

Esta clase recorre el AST y genera un archivo `.dot`.

Luego utiliza Graphviz para convertirlo a imagen `.png`.

Archivos generados:

```
reportes/ast.dot  
reportes/ast.png
```

El comando usado internamente es similar a:

```
dot -Tpng reportes/ast.dot -o reportes/ast.png
```

20. Interfaz gráfica

La interfaz gráfica se encuentra en:

```
com.golite.gui
```

La clase principal es:

```
MainWindow
```

Esta clase contiene:

- Editor.
- Consola.
- Menú Archivo.
- Menú Ejecutar.
- Menú Reportes.
- Tabla de tokens.
- Reporte de errores.
- Tabla de símbolos.

- Visualización del AST.
-

21. Compilación del proyecto

Para compilar:

```
mvn clean compile
```

22. Generación del JAR

Para generar el JAR con dependencias:

```
mvn clean package
```

El archivo generado se encuentra en:

```
target/GoLite-1.0-SNAPSHOT-jar-with-dependencies.jar
```

23. Ejecución del JAR

```
java -jar target/GoLite-1.0-SNAPSHOT-jar-with-dependencies.jar
```

24. Consideraciones técnicas

- El proyecto utiliza Java JDK 21.
 - El AST se genera correctamente si Graphviz está instalado y configurado en el PATH.
 - Los warnings generados por CUP no impiden la ejecución mientras Maven finalice con `BUILD SUCCESS`.
 - El archivo `.jar` con dependencias es el que debe usarse para ejecutar el proyecto sin depender de Maven.
 - La gramática BNF documenta la sintaxis soportada por esta implementación de GoLite.
-

25. Prueba técnica recomendada

```
type Persona struct {  
    nombre string, edad int  
}  
  
func cambiarNombre(p Persona, nuevo string) {  
    p.nombre = nuevo  
}  
  
func main() {  
    p := Persona{nombre: "Dalio", edad: 24}  
  
    fmt.Println(p.nombre)  
  
    cambiarNombre(p, "Carlos")  
  
    fmt.Println(p.nombre)  
}
```

Salida esperada:

```
Dalio  
Carlos
```